

CQRS & Clean Architecture, by example

A guided walkthrough of one request — `POST /api/data-sources` — through the SalesCom service

This document follows a **single HTTP request** from the moment it hits the controller until the JSON leaves the server, naming every type it passes through and explaining *why* each piece exists. By the end you should be able to read any feature in this codebase — and add your own — with confidence.

What you'll learn

1. Three ideas in one minute
2. The four layers & the dependency rule
3. The cast of characters
4. The request, traced step by step
5. The whole flow on one page
6. What happens when things go wrong
7. Why build it this way?
8. Add your own command (cheat sheet)

The example endpoint. `POST /api/data-sources` registers a source table plus its columns in one transaction. We picked it because it exercises *the entire pipeline*: routing & model-binding, the custom command dispatcher, the decorator chain (logging → validation → handler), the unit-of-work + repository, entity-to-DTO mapping, the `Result` type, and the unified `ApiResponse` envelope.

1. Three ideas in one minute

Clean Architecture

Code is split into four projects arranged in concentric rings. **Dependencies only ever point inward.** The business rules (Domain) know nothing about the database, HTTP, or JSON. The outer rings depend on the inner ones, never the reverse. This is what lets you swap Postgres for something else, or test a use-case with no web server.

CQRS — Command/Query Responsibility Segregation

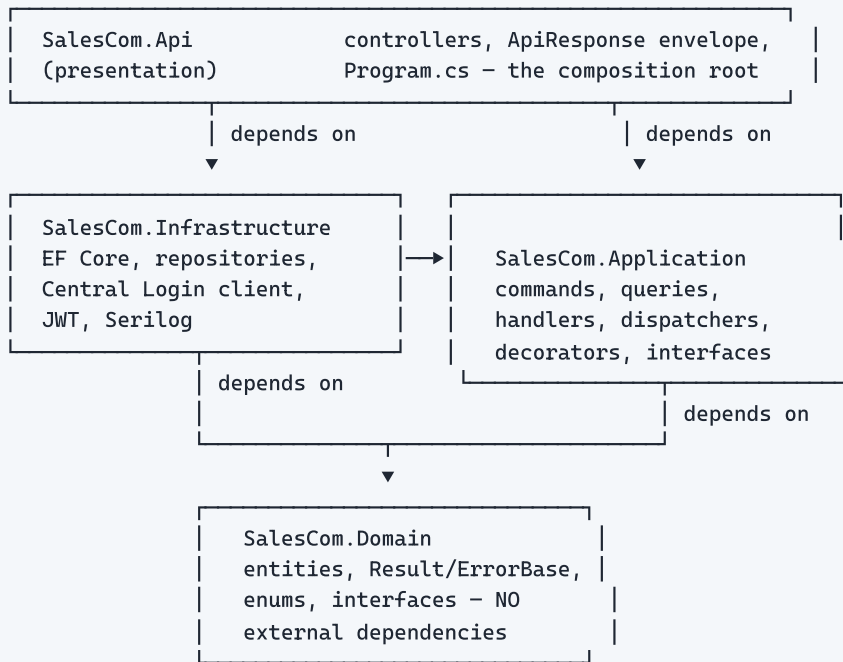
Every use-case is modelled as either a **Command** (it changes state — "create a data source") or a **Query** (it reads state — "list data sources") . Each gets its own little message class and its own handler. There is no "service" class with twelve methods; there is one class per thing-the-system-can-do. In this project a use-case lives in a **vertical slice**: a folder holding the Command, its Handler, and its Validator side by side.

A custom "mediator"

The controller never news-up a handler. It hands the command to a **dispatcher**, which finds the right handler and invokes it. Around every handler sits a **decorator chain** (logging, then validation) so cross-cutting concerns are written once, not in every handler. This is the same idea as the MediatR library — but here it is ~30 lines of our own code, so nothing is hidden.

How they fit together: Clean Architecture says *where* code lives; CQRS says *how a use-case is shaped*; the dispatcher + decorators are the *plumbing* that connects a controller to a use-case without coupling them.

2. The four layers & the dependency rule



Layer	Owns	Knows about
Domain	Entities (<code>DataSource</code>), <code>Result</code> / <code>ErrorMessage</code> , enums, repository <i>interfaces</i>	Nothing else — no EF, no ASP.NET
Application	Commands, queries, handlers, validators, the dispatcher + decorators, and <i>interfaces</i> for things it needs (e.g. <code>IUnitOfWork</code> , <code>IClock</code>)	Domain only
Infrastructure	<i>Implementations</i> : EF <code>DbContext</code> , the unit of work / repositories, the Central Login HTTP client, JWT, Serilog	Application + Domain
Api	Controllers, the <code>ApiResponse</code> mapping, middleware, <code>Program.cs</code> (wires everyone together)	Application + Infrastructure

The key trick — dependency inversion. The handler (Application) needs to save to a database, but Application must not reference EF Core. So Application *declares an interface* it wants (`IUnitOfWork` in Domain), and Infrastructure *implements* it. At runtime the DI container injects the Infrastructure implementation. The arrow of *compilation* points inward; the arrow of *control* at runtime points outward. That inversion is the whole game.

3. The cast of characters

Eleven types take part in our one request. Skim this table now; we'll meet each one in order.

#	Type	Layer / file	Job
1	<code>DataSourceesController</code>	Api · <code>Controllers/</code>	Receives HTTP, hands the command to the dispatcher, maps the result back to HTTP.
2	<code>CreateDataSourceCommand</code>	Application · <code>Commands/.../CreateDataSource/</code>	The request, as a typed message. <i>Is</i> the API contract.
3	<code>ICommandDispatcher</code> / <code>CommandDispatcher</code>	Application · <code>Messaging/</code>	Finds the handler for a command and invokes it (our "mediator").
4	<code>AddApplication</code> (DI)	Application · <code>DependencyInjection.cs</code>	At startup, wraps each handler in the decorator chain and registers it.
5	<code>LoggingCommandHandlerDecorator</code>	Application · <code>Behaviours/</code>	Outermost wrapper — logs entry/exit/duration.
6	<code>ValidationCommandHandlerDecorator</code>	Application · <code>Behaviours/</code>	Runs <code>FluentValidation</code> ; short-circuits to a failed <code>Result</code> if invalid.
7	<code>CreateDataSourceValidator</code>	Application · same slice	The rules for a valid command.
8	<code>CreateDataSourceHandler</code>	Application · same slice	The actual use-case logic.
9	<code>IUnitOfWork</code> / <code>IGenericRepository<T></code>	Domain interface · Infra impl	Persistence, behind an interface.
10	<code>Result<T></code> / <code>ErrorBase</code>	Domain · <code>Common/</code>	Success-or-failure return type (no exceptions for business rules).
11	<code>ToApiResponse</code> + <code>ApiResponse<T></code>	Api / Application	Turns a <code>Result</code> into an HTTP status + the unified JSON envelope.

4. The request, traced step by step

A client sends:

```
POST /api/data-sources
Authorization: Bearer <jwe access token>
Content-Type: application/json

{
  "sourceTableName": "ev_recharge_com",
  "aliasTableName": "EV Recharge",
  "tableDescription": "Primary recharge feed",
  "isActive": true,
  "columns": [
    { "columnName": "channel_type", "aliasColumnName": "Channel Type", "dataType": "character varying" }
  ]
}
```

1 Routing, auth & model binding → the controller action

src/SalesCom.Api/Controllers/DataSourcesController.cs

ASP.NET routes `POST /api/data-sources` to `CreateAsync`. Before the body runs, two framework steps happen: the `[HasPermission(...)]` attribute checks (against the database) that the caller may *Manage* data sources, and the JSON body is **bound directly into the command object** — there is no separate "request DTO"; **the command is the wire contract**.

```
[HttpPost]
[HasPermission(Permissions.DataSources.Manage)]
[ProducesResponseType(typeof(ApiResponse<DataSourceResponse>), StatusCodes.Status201Created)]
[ProducesResponseType(typeof(ApiResponse), StatusCodes.Status400BadRequest)]
[ProducesResponseType(typeof(ApiResponse), StatusCodes.Status409Conflict)]
public async Task<IActionResult> CreateAsync(
    [FromBody] CreateDataSourceCommand command,
    CancellationToken cancellationToken)
{
    var result = await commandDispatcher.DispatchAsync(command, cancellationToken);
    return result.ToApiResponse(this, "Data source created.", StatusCodes.Status201Created);
}
```

Notice how little the controller does. It does not validate, does not touch the database, does not build the response shape. It binds → dispatches → maps. Two collaborators are injected via the constructor:

`ICommandDispatcher` and `IQueryDispatcher`. That's the whole controller's surface.

2 The command — a typed message that *is* the use-case

src/SalesCom.Application/Commands/DataSources/CreateDataSource/CreateDataSourceCommand.cs

```
public sealed record CreateDataSourceCommand(
    string SourceTableName,
    string AliasTableName,
    string? TableDescription,
    bool IsActive,
    IReadOnlyList<CreateDataSourceColumn> Columns) : ICommand<Result<DataSourceResponse>>;
```

```
public sealed record CreateDataSourceColumn(string ColumnName, string AliasColumnName, string DataType);
```

It is an immutable **record**. The important part is the interface it implements:

ICommand<Result<DataSourceResponse>>. That single line says three things: "I am a command" (state-changing), "dispatch me", and "when handled I produce a **Result<DataSourceResponse>**." The marker interfaces are deliberately empty:

src/SalesCom.Application/Messaging/ICommand.cs · ICommandHandler.cs

```
public interface ICommand<TResponse> { } // a command that yields TResponse
public interface ICommand : ICommand<Result> { } // a command with no payload, just
success/failure

public interface ICommandHandler<in TCommand, TResponse>
    where TCommand : ICommand<TResponse>
{
    Task<TResponse> HandleAsync(TCommand command, CancellationToken cancellationToken);
}
```

Why a command instead of method parameters? Because a single typed object can be passed through the dispatcher, logged by name, validated generically, and serialized straight from the HTTP body. The command's *type* is the key that selects the right handler.

3 The dispatcher finds and invokes the handler

src/SalesCom.Application/Messaging/CommandDispatcher.cs

```
public Task<TResponse> DispatchAsync<TResponse>(ICommand<TResponse> command, CancellationToken
cancellationToken)
{
    ArgumentNullException.ThrowIfNull(command);

    // Build the closed handler type for THIS command's runtime type, e.g.
    // ICommandHandler<CreateDataSourceCommand, Result<DataSourceResponse>>
    var handlerType = typeof(ICommandHandler<,>).MakeGenericType(command.GetType(), typeof(TResponse));
    var handler = serviceProvider.GetRequiredService(handlerType); // ask the DI container for it
    var method = handlerType.GetMethod(nameof(ICommandHandler<ICommand<TResponse>,
TResponse>.HandleAsync));
    return (Task<TResponse>)method.Invoke(handler, [command, cancellationToken]);
}
```

This is the "mediator". It uses the command's **runtime type** to construct the matching **ICommandHandler<,>** type, asks the DI container for whatever is registered against it, and invokes **HandleAsync** by reflection. The controller and the handler never reference each other.

What the container hands back is not the bare handler — it's the handler already wrapped in the decorator chain. That wrapping was decided once, at startup (next step).

4 Startup wiring — how the decorator chain was assembled

src/SalesCom.Application/DependencyInjection.cs

At boot, **AddApplication()** reflection-scans the assembly for every **ICommandHandler<,>** and registers, against that interface, a factory that builds the onion: **Logging(Validation(realHandler))**.

```

services.TryAddTransient(implementation); // the real CreateDataSourceHandler

var loggingDecorator = typeof(LoggingCommandHandlerDecorator<,>).MakeGenericType(commandType,
responseType);
var validationDecorator = typeof(ValidationCommandHandlerDecorator<,>).MakeGenericType(commandType,
responseType);

services.AddTransient(service, sp =>
{
    var real = sp.GetRequiredService(implementation); // innermost
    var validation = ActivatorUtilities.CreateInstance(sp, validationDecorator, real);
    var logging = ActivatorUtilities.CreateInstance(sp, loggingDecorator, validation); // outermost
    return logging;
});

```

Why decorators? Logging and validation must happen for every command. Rather than copy that code into 20 handlers, each concern is a class that implements the same `ICommandHandler<,>` interface and holds a reference to the next one in. Because every layer has the same shape, they stack like Russian dolls. The decorator order is fixed here: **Logging** → **Validation** → **Handler**.

5 Decorator 1 (outermost): logging

src/SalesCom.Application/Behaviours/LoggingCommandHandlerDecorator.cs

```

public async Task<TResponse> HandleAsync(TCommand command, CancellationToken cancellationToken)
{
    var commandName = typeof(TCommand).Name;
    using var scope = logger.BeginScope(/* CommandName, CommandType */);

    logger.LogInformation("Handling command {CommandName}", commandName);
    var sw = Stopwatch.StartNew();
    try
    {
        var response = await inner.HandleAsync(command, cancellationToken); // — call inward →
        logger.LogInformation("Handled command {CommandName} in {ElapsedMs} ms", commandName,
sw.ElapsedMilliseconds);
        return response;
    }
    catch (Exception ex)
    {
        logger.LogError(ex, "Command {CommandName} failed after {ElapsedMs} ms", commandName,
sw.ElapsedMilliseconds);
        throw;
    }
}

```

It logs, starts a stopwatch, then calls `inner.HandleAsync` — which is the validation decorator. Control will return here last, so it also logs the total duration. `inner` is just another `ICommandHandler<TCommand, TResponse>`; the decorator doesn't know or care that it's the validator.

6 Decorator 2: validation (the short-circuit)

src/SalesCom.Application/Behaviours/ValidationCommandHandlerDecorator.cs

```

public async Task<TResponse> HandleAsync(TCommand command, CancellationToken cancellationToken)
{
    if (validators.Any())
    {

```

```

        var context = new ValidationContext<TCommand>(command);
        var failures = (await Task.WhenAll(validators.Select(v => v.ValidateAsync(context,
cancellationToken))))
            .SelectMany(r => r.Errors).Where(f => f is not null).ToList();

        if (failures.Count > 0)
        {
            var error = ErrorBase.Validation($"Validation.{failures[0].PropertyName}",
failures[0].ErrorMessage);
            // bridge the error into the generic Result<T> the dispatcher expects, then RETURN — handler
never runs
            return /* Result.Failure<DataSourceResponse>(error) as TResponse */;
        }
    }
    return await inner.HandleAsync(command, cancellationToken); // — valid → call the real handler
→
}

```

All `IValidator<CreateDataSourceCommand>` instances are injected (FluentValidation found them at startup). If any rule fails, the decorator builds a failed `Result` and **returns immediately** — the real handler is never reached, and no exception is thrown. If everything passes, it calls inward to the handler.

src/SalesCom.Application/Commands/DataSources/CreateDataSource/CreateDataSourceValidator.cs (step 7)

```

RuleFor(x => x.SourceTableName).NotEmpty().MaxLength(200);
RuleFor(x => x.AliasTableName).NotEmpty().MaxLength(200);
RuleFor(x => x.Columns).NotEmpty().WithMessage("At least one column is required.");
RuleForEach(x => x.Columns).SetValidator(new CreateDataSourceColumnValidator());

```

This is why handlers are clean. By the time control reaches the handler, the command is *guaranteed* structurally valid. The handler only worries about *business* rules (e.g. "is this table already registered?"), never "is this field empty?".

8 The handler — the actual use-case

src/SalesCom.Application/Commands/DataSources/CreateDataSource/CreateDataSourceHandler.cs

```

internal sealed class CreateDataSourceHandler(
    IUnitOfWork unitOfWork,
    IClock clock) : ICommandHandler<CreateDataSourceCommand, Result<DataSourceResponse>>
{
    public async Task<Result<DataSourceResponse>> HandleAsync(
        CreateDataSourceCommand command, Cancellation token cancellationToken)
    {
        var sourceTable = command.SourceTableName.Trim();
        var sources = unitOfWork.Repository<DataSource>(); // ① get a repo from the unit of
work

        var lowered = sourceTable.ToLower();
        if (await sources.AnyAsync(d => d.SourceTableName.ToLower() == lowered, cancellationToken))
            return DataSourceErrors.AlreadyRegistered; // ② business rule → failed
Result (409)

        var now = clock.UtcNow; // ③ time via an interface, not
DateTime.Now
        var dataSource = new DataSource
        {
            SourceTableName = sourceTable,
            AliasTableName = command.AliasTableName.Trim(),

```



```

        TableDescription = command.TableDescription?.Trim(),
        IsActive         = command.IsActive,
        CreatedOnUtc     = now,
    };
    await sources.AddAsync(dataSource, cancellationToken); // ④ stage the parent

    var columns = command.Columns.Select(c => new DataSourceColumn
    {
        DataSourceId      = dataSource.Id,
        ColumnName        = c.ColumnName.Trim(),
        AliasColumnName    = c.AliasColumnName.Trim(),
        DataType           = c.DataType.Trim(),
        CreatedOnUtc       = now,
    }).ToList();
    await unitOfWork.Repository<DataSourceColumn>().AddRangeAsync(columns, cancellationToken); // ⑤
    stage children

    await unitOfWork.SaveChangesAsync(cancellationToken); // ⑥ ONE commit = one
    transaction
    return dataSource.ToResponse(columns); // ⑦ map entity → DTO (implicit
    success Result)
    }
}

```

Read the seven numbered moves. Two things deserve attention:

- **It depends only on interfaces** — `IUnitOfWork` and `IClock`, both declared in Domain. It has no idea EF Core or Postgres exist. The concrete `UnitOfWork` is injected from Infrastructure.
- **One `SaveChangesAsync`** commits the parent and all child columns together. If anything throws, nothing is written — the handler is all-or-none.

`src/SalesCom.Domain/Interfaces/IUnitOfWork.cs` (the contract the handler leans on)

```

public interface IUnitOfWork
{
    IGenericRepository<T> Repository<T>() where T : class; // a typed repo for any aggregate
    ISqlRepository Sql { get; }
    Task<int> SaveChangesAsync(Cancellation token cancellationToken);
}

```

Why route persistence through an interface in Domain? So the use-case stays testable and database-agnostic. A unit test injects a fake `IUnitOfWork`; production injects the EF-backed one. The Application project literally cannot compile a reference to EF Core — the architecture enforces the boundary.

9 Mapping the entity to a response DTO

`src/SalesCom.Application/Mappings/DataSourceMappingExtensions.cs`

```

public static DataSourceResponse ToResponse(this DataSource source, IEnumerable<DataSourceColumn>
columns) =>
    new(source.Id, source.SourceTableName, source.AliasTableName, source.TableDescription,
        source.IsActive, columns.Select(c => c.ToResponse()).ToList(),
        source.CreatedOnUtc, source.UpdatedAtUtc);

```

Entities are plain data classes with no behaviour; mapping to the DTO is an **extension method** that lives with the feature, never on the entity. `DataSourceResponse` is the read-model the client actually receives — it deliberately omits internal fields (like the EF concurrency token).

10 The return type: `Result<T>`, not exceptions

`src/SalesCom.Domain/Common/Result.cs`

Notice the handler `return` s a bare `DataSourceErrors.AlreadyRegistered` in one place and a bare `DataSourceResponse` in another — yet the method returns `Result<DataSourceResponse>`. That works because of two **implicit conversions**:

```
public sealed class Result<TValue> : Result
{
    public static implicit operator Result<TValue>(TValue value) => Success(value);    // value  →
    success
    public static implicit operator Result<TValue>(ErrorBase error) => Failure<TValue>(error); // error
    → failure
}
```

So business outcomes — success or a named failure like `AlreadyRegistered` (a `Conflict` error) — travel home through the normal return channel. Exceptions are reserved for genuine infrastructure faults. An `ErrorBase` carries a stable `Code`, a human `Message`, and a `Type` (Validation / NotFound / Conflict / Unauthorized / Forbidden / Unexpected) that decides the HTTP status later.

11 Unwinding the chain & mapping `Result` → HTTP

The `Result` now travels back *outward*: the validation decorator returns it untouched (it only acts on the way in), and the logging decorator records the elapsed time and returns it to the dispatcher, which returns it to the controller. The controller's final line converts it to an HTTP response:

`src/SalesCom.Api/Extensions/ResultExtensions.cs`

```
public static IActionResult ToApiResponse<T>(this Result<T> result, ControllerBase controller,
    string successMessage = "Success", int successStatus = StatusCodes.Status200OK)
{
    if (result.IsSuccess)
        return controller.StatusCode(successStatus, ApiResponse<T>.Ok(result.Value, successMessage));

    var status = MapStatus(result.Error.Type);    // Conflict → 409, NotFound → 404, Validation → 400, ...
    return controller.StatusCode(status, ApiResponse<T>.Fail(result.Error.Message, result.Error.Code));
}
```

The error *type* drives the status code, and the body is always the same unified envelope, so every endpoint — success or failure — returns one predictable shape.

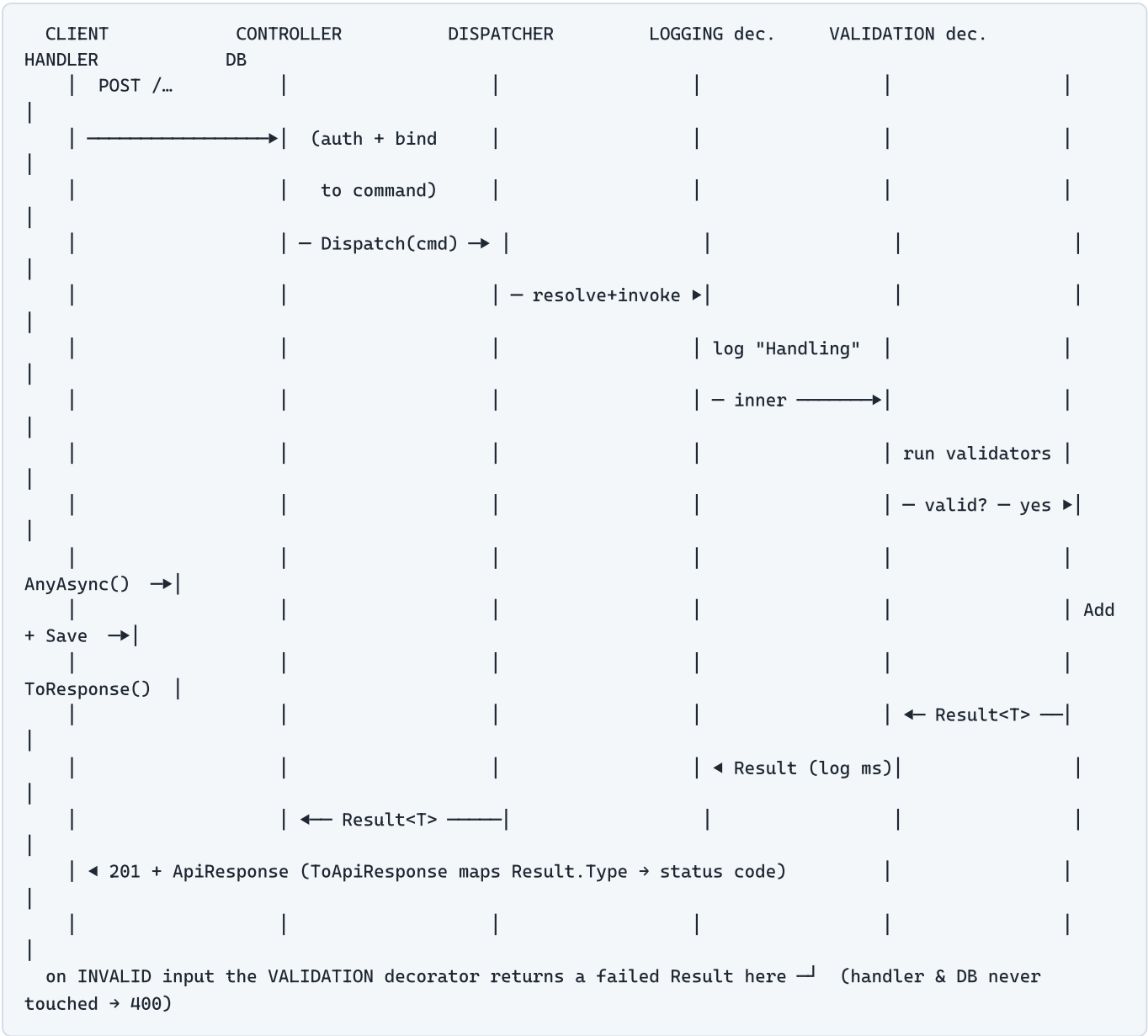
12 The HTTP response

On success the client receives **201 Created** and:

```
{
  "success": true,
  "message": "Data source created.",
  "data": {
    "id": "7b1f...",
    "sourceTableName": "ev_recharge_com",
    "aliasTableName": "EV Recharge",
    "isActive": true,
    "columns": [ { "id": "...", "columnName": "channel_type", "aliasColumnName": "Channel Type",
    "dataType": "character varying" } ],
    "createdOnUtc": "2026-06-13T12:00:00+00:00",
    "updatedAtUtc": null
  }
}
```

```
}  
}
```

5. The whole flow on one page



6. What happens when things go wrong

Same pipeline, different exits. The `Error.Type` set deep in the stack becomes the HTTP status at the top — the controller code does not change.

Scenario	Where it's caught	Returns	HTTP
Empty <code>sourceTableName</code> / no columns	Validation decorator (step 6)	<code>ErrorBase.Validation("Validation.SourceTableName", ...)</code>	400
Table already registered	Handler business rule (step 8②)	<code>DataSourceErrors.AlreadyRegistered</code> (Conflict)	409
Caller lacks <code>DataSources.Manage</code>	<code>[HasPermission]</code> before the action	authorization envelope	403
No / invalid bearer token	JWT middleware before routing	authentication envelope	401
DB unreachable, bug, etc.	<code>GlobalExceptionHandler</code> (last resort)	<code>Server.UnexpectedError</code>	500

Two failure channels, by design. Expected, business outcomes flow through `Result` (no stack-trace cost, explicit in the signature). Truly *exceptional* faults throw and are caught once, globally. You almost never write `try/catch` in a handler.

7. Why build it this way?

Q Why not just call a service class from the controller?

Because a "DataSourceService" accumulates many methods and many dependencies, and every method re-implements logging, validation, and transaction handling. One-command-one-handler keeps each use-case small, independently testable, and honest about exactly what it needs (its constructor lists its dependencies).

Q Why a custom dispatcher instead of MediatR?

The whole mechanism is ~30 lines you can read in one sitting (steps 3–4). There is no magic, no extra package to keep current, and the team controls the decorator order exactly. The trade-off — a little reflection in the dispatcher — is paid once and centrally.

Q Why does the Domain own the repository interface but not the implementation?

This is dependency inversion (section 2). It keeps the dependency arrows pointing inward so the business logic never takes a hard dependency on EF Core, and so a test can swap in a fake. Infrastructure supplies the real implementation at runtime through DI.

Q Why **Result** instead of throwing?

"This table is already registered" is an *expected* outcome, not a crash. Modelling it as a value makes it visible in the method signature, free of stack-unwinding cost, and trivially mappable to a 409. Exceptions stay rare and genuinely exceptional.

Q Why is the command both the API request and the internal message?

Fewer types, no hand-written request→command copying, and the command name is a perfect logging/telemetry key. The trade-off is that the wire shape is coupled to the command; this project accepts that deliberately.

8. Add your own command (cheat sheet)

To add, say, `POST /api/data-sources/{id}/archive` :

1. Create folder `Application/Commands/DataSources/ArchiveDataSource/`.
2. Add `ArchiveDataSourceCommand : ICommand<Result>` (a `record`).
3. Add `ArchiveDataSourceHandler : ICommandHandler<ArchiveDataSourceCommand, Result>` — inject `IUnitOfWork`, do the work, `SaveChangesAsync` once, return `Result.Success()` or a named error.
4. (Optional) Add `ArchiveDataSourceValidator : AbstractValidator<ArchiveDataSourceCommand>` in the same folder.
5. Add the controller action: bind the command, `commandDispatcher.DispatchAsync( ... )`, `result.ToApiResponse(this)`.

That's it — no registration step. `AddApplication()` 's reflection scan (step 4) discovers the new handler and validator automatically and wraps the handler in Logging → Validation. You write the use-case; the pipeline is already there.

Generated from the SalesCom codebase · example flow: `POST /api/data-sources` → `CreateDataSourceCommand` → `CommandDispatcher` → Logging → Validation → `CreateDataSourceHandler` → `IUnitOfWork` → `Result<DataSourceResponse>` → `ApiResponse<DataSourceResponse>`. Code excerpts are quoted verbatim (lightly trimmed where noted with `...`).